# 2026-11-06 글 알림

- 날짜: 2026-11-06
- 앱: `lgcns-fe-next`
- 기준 코드: `prisma/schema.prisma`의 `Notification`, `service/postService.ts`의 `notifyAuthor`, `service/notificationService.ts`, `service/notificationWatch.ts`, `app/api/notifications/stream/route.ts`, `component/molecules/NotificationStream.tsx`, `component/organisms/NotificationMenu.tsx`, `app/AppHeader.tsx`
- 이 문서는 그 파일이 지금 디스크에 있는 동작만 적는다. 5초마다 알림 목록을 다시 읽는 서버 액션 `listNotifications`는 없다. 푸시 알림, 이메일, 웹소켓도 이 경로에 없다.

다른 사람이 내 글에 좋아요를 누르거나 댓글을 달면 `Notification` 행이 생긴다. 헤더는 그 목록을 서버에서 한 번 그린다. 그 다음 브라우저의 `EventSource`가 `/api/notifications/stream`을 연다. 서버는 MongoDB change stream으로 그 컬렉션의 변경을 받아, 같은 수신자에게만 SSE로 보낸다. 화면을 연 뒤에 새로 도착한 읽지 않은 알림만 토스트로 그린다.

## 1. 파일이 언제 실행되는가

`app/layout.tsx`에는 알림 코드를 불러오는 `<script>`가 없다. `async`, `defer`, `type="module"` 속성을 단 스크립트 태그도 없다. 같은 레이아웃의 `ServiceWorkerRegister`는 프로덕션에서만 `/sw.js`를 등록한다. `NotificationMenu`는 그 등록을 호출하지 않는다.

| 파일 | 선언 | 실행 시점 | async / defer / type="module"과의 차이 |
|---|---|---|---|
| `prisma/schema.prisma` | `model Notification` | Prisma가 스키마로 클라이언트를 만들 때 쓰인다. 브라우저가 이 파일을 실행하지 않는다 | 스크립트가 아니다 |
| `service/postService.ts` | `async function notifyAuthor`. export는 `postService` 객체다. 이 함수 자체는 export되지 않는다 | 서버에서 `toggleLike`이 좋아요를 새로 만들 때, `createComment`가 댓글을 만든 뒤에 호출된다 | 함수의 `async`는 ECMAScript 비동기 함수다. HTML `async` 속성이 아니다 |
| `service/notificationItem.ts` | `export function toNotificationItem` | 목록 조회와 change stream이 행을 항목으로 바꿀 때 서버에서 호출된다 | 브라우저 모듈 그래프에 들어가지 않는다 |
| `service/notificationService.ts` | `async function listForUser`, `markRead`, `markAllRead`. `function subscribe`, `whenReady`. `export const notificationService` | 서버 렌더, 서버 액션, SSE 라우트가 호출할 때 Prisma와 change stream 구독을 실행한다 | 브라우저가 이 파일을 내려받지 않는다 |
| `service/notificationWatch.ts` | `function subscribe`, `async function connect`. `export const notificationWatch` | SSE 구독이 있을 때 서버 프로세스에서 컬렉션 `Notification`의 change stream 하나를 연다 | Node에서 실행된다. `<script type="module">`이 아니다 |
| `lib/readNotificationSnapshot.ts` | `export async function readNotificationSnapshot` | `AppHeader`의 `HeaderData`가 서버에서 기다린다. 세션이 없으면 `null`이다 | 서버 렌더 중의 비동기 함수다 |

| 파일 | 선언 | 실행 시점 | async / defer / type="module"과의 차이 |
|---|---|---|---|
| actions/notificationActions.ts | 파일 첫 줄 "use server". export async function markNotificationRead, markAllNotificationsRead | 메뉴가 항목을 열거나 모두 읽음을 누를 때 서버에서 실행된다. 목록을 주기적으로 읽지 않는다 | "use server"는 번들러 지시문이다. <script>의 async가 아니다 |
| app/api/notifications/stream/route.ts | export async function GET. 본문 첫 호출은 await connection() | EventSource의 GET이 살아 있는 동안 text/event-stream 본문을 이어서 쓴다 | 응답 본문이 이벤트 스트림이다. defer 스크립트가 아니다 |
| app/AppHeader.tsx | async function HeaderData. export default function AppHeader | (main)과 (post) 레이아웃이 서버에서 헤더를 그릴 때 실행된다. 조회는 Suspense 자식 안에 있다 | 서버 컴포넌트다 |
| component/templates/Header.tsx | export default function Header. "use client" 없음 | 서버 컴포넌트로 헤더를 그린다. notifications, markRead, markAll을 props로 받아 NotificationMenu에 넘긴다 | 서버 컴포넌트다. 브라우저가 이 파일을 defer로 실행하지 않는다 |
| component/organisms/NotificationMenu.tsx | 파일 첫 줄 "use client". export default function NotificationMenu | 서버가 헤더 HTML을 만든 뒤, 브라우저가 클라이언트 번들을 받아 hydrate할 때 함수가 다시 실행된다 | "use client"는 <script type="module">이 아니다. 이 파일 경로를 src로 받는 태그는 없다 |
| component/molecules/NotificationStream.tsx | 파일 첫 줄 "use client". useEffect 안에서 new EventSource | hydrate된 뒤, 탭이 visible일 때 /api/notifications/stream을 연다. 숨으면 close한다 | EventSource는 DOM API다. 스크립트 로드 속성으로 붙지 않는다 |

HTML에서 async 스크립트는 받는 즉시 실행하고 문서 파싱을 막지 않으며 순서가 없다. defer는 파싱을 막지 않고, 문서 파싱이 끝난 뒤 적힌 순서대로 실행한다. type="module"은 그 지연 실행에 import/export가 더해진다. 이 알림 경로는 그 세 속성으로 붙지 않는다. 행을 만드는 코드는 서버 요청 안에서 실행된다. 배지와 토스트는 hydrate 이후 EventSource의 onmessage에서 갱신된다.

참고: HTML script 요소, MDN script, ECMA-262 비동기 함수, HTML 서버 전송 이벤트, MDN EventSource, Next.js 서버 컴포넌트와 클라이언트 컴포넌트.

## 2. 행이 생기는 조건

notifyAuthor는 authorId와 actorId가 같으면 바로 반환한다. 내 글에 내가 누른 좋아요와 내가 단 댓글은 알림이 되지 않는다.

좋아요는 toggleLike이 기존 행이 없을 때만 만든다. 글의 authorId를 찾은 뒤에 kind: "like"로 notification.create를 한다. 이미 있으면 좋아요를 지우고, 같은 actorId, postId, kind: "like" 알림을 deleteMany로 지운다. 댓글은 comment.create 다음에 kind: "comment"로 만든다. 글을 삭제하면 그 postId의 알림도 deleteMany로 지운다.

readAt은 만들 때 넣지 않는다. 그래서 새 행은 읽지 않은 상태다.

이 함수는 서버 액션 actions/postActions.ts의 toggleLike, createComment와 app/api/posts/[postId]/like, comments 라우트가 postService를 호출할 때 같이 실행된다. 알림을 만드는 전용 라우트는 없다. 전달 전용 라우트는 GET /api/notifications/stream뿐이다.

## 3. 처음 그리는 목록

app/(main)/layout.tsx와 app/(post)/layout.tsx는 AppHeader를 둔다. HeaderData는 readNotificationSnapshot을 기다린다. 세션이 없으면 null이고, 있으면 notificationService.listForUser를 호출한다.

listForUser는 Promise.all로 두 쿼리를 같이 보낸다. findMany는 recipientId가 현재 사용자인 행을 createdAt 내림차순으로 20개만 가져온다. count는 그 사용자의 readAt이 null인 행 전체다. 배지 숫자는 화면에 나열된 20개가 아니라 이 count다. 20개를 넘으면 버튼에는 9+로 줄인다.

toNotificationItem은 kind가 "like"도 "comment"도 아니면 null을 반환하고, 그 행은 목록에서 빠진다. read는 readAt !== null이다. createdAt은 ISO 문자열이다.

조회는 Suspense 안에 있다. fallback 헤더는 notifications={null}이라 NotificationMenu가 null을 반환한다. 로그인하지 않아도 메뉴는 그리지 않는다. 헤더 버튼의 문구는 알림이다.

Header와 NotificationMenu는 @/service를 import하지 않는다. 읽음 액션은 organism이 props로 받는다.

참고: MDN Promise.all, React Suspense.

## 4. SSE가 하는 일

NotificationStream은 enabled일 때 effect에서 연결한다. 메뉴는 로그인되어 있고 notifications가 있을 때만 이 컴포넌트를 그리므로, 그 경우에만 enabled다. 탭이 document.visibilityState === "visible"일 때만 new EventSource("/api/notifications/stream")을 연다. 숨으면 close하고, 다시 보이면 새로 연다. 언마운트할 때도 닫는다. onmessage의 data를 JSON.parse해서 type이 있는 객체만 onEvent로 넘긴다.

라우트는 connection() 다음 readCurrentUser를 본다. 사용자가 없으면 401이고 본문이 없다. 있으면 Content-Type: text/event-stream; charset=utf-8인 ReadableStream을 반환한다. Cache-Control은 no-cache, no-transform이고, X-Accel-Buffering은 no다.

보내는 순서는 이렇다.

1. retry: 10000. 브라우저가 끊긴 뒤 다시 연결하기 전에 기다리는 밀리초다.
2. notificationService.subscribe로 이 사용자의 리스너를 등록한다. 스냅샷 전에 도착한 변경은 배열에 둔다.
3. whenReady로 change stream이 열리기를 기다린다. 연 뒤에 listForUser 결과로 type: "snapshot"을 보낸다.
4. 모아 둔 변경을 스냅샷 다음에 보낸다.
5. 25초마다 : ping 주석을 보낸다. 주석은 EventSource가 onmessage로 올리지 않는다.

요청이 abort되거나 스트림이 cancel되면 구독을 해제하고 주석 간격을 지운다. 이 간격은 목록을 다시 읽는 주기가 아니다. 연결을 유지하는 주석이다.

참고: HTML 서버 전송 이벤트, MDN 서버 전송 이벤트, Next.js connection, Next.js 라우트 핸들러.

## 5. change stream과의 관계

notificationWatch는 lib/mongo.ts의 MongoClient로 컬렉션 Notification을 watch한다. 옵션은 fullDocument: "updateLookup", fullDocumentBeforeChange: "whenAvailable"이다. 구독자가 있는 동안 프로세스당 커서 하나다. SSE 연결마다 커서를 만들지 않고, recipientId가 같은 리스너에게만 이벤트를 넘긴다. 허브는 globalThis에 둔다.

| 변경 | SSE type | 메뉴가 하는 일 |
|---|---|---|
| insert | insert와 item | 같은 id가 없으면 목록 앞에 넣고 20개만 남긴다. 안 읽음이면 배지를 1 더한다. 처음 보는 id면 토스트 |
| update의 updatedFields에 readAt | read와 id | 그 항목을 읽음으로 바꾸고, 안 읽음이었으면 배지를 1 줄인다 |
| readAt이 날짜인 replace | read와 id | 위와 같다 |
| delete | delete와 id | 목록에 있는 항목만 뺀다. 안 읽음이었으면 배지를 1 줄인다 |

삭제의 수신자는 fullDocumentBeforeChange.recipientId다. 시작할 때 collMod로 changeStreamPreAndPostImages를 켜려고 한다. 그 명령이 실패하면 imagesEnabled를 올리지 않고, 삭제 문서에 수신자가 없으면 그 삭제는 보내지 않는다. readAt이 없는 update와 replace는 보내지 않는다.

스트림이 오류나 close로 끊기면 구독자가 남아 있을 때 5초 뒤 다시 watch한다. 이 5초는 브라우저가 목록을 다시 요청하는 간격이 아니다. 서버 커서를 다시 여는 간격이다. 실패 로그 알림 변경 스트림을 열지 못했습니다.는 한 번만 남기고, 연결 문자열은 로그에서 지운다. 다시 열리면 그 플래그를 내린다.

이 커서는 레플리카 셋에서만 동작한다. 단독 MongoDB에서는 change stream이 열리지 않는다. 그때도 첫 화면의 서버 렌더 목록은 남아 있다. whenReady는 연결이 실패해도 열린 promise를 끝내므로, 라우트는 스냅샷을 보낼 수 있다.

참고: MongoDB change streams.

## 6. 토스트와 읽음

seenRef는 Set이거나 null이다. 첫 이벤트에서 null이면, 서버가 처음 그린 항목 id로 Set을 만든다. 스냅샷의 항목 id는 모두 이 Set에 넣는다. 읽지 않았고 Set에 없던 id만 토스트로 띄운다. 그래서 새로고침 직후 이미 화면에 있던 알림은 배지로만 남고, 그 사이에 새로 생긴 안 읽은 알림은 토스트가 된다.

insert도 읽지 않았고 Set에 없는 id만 토스트로 띄운다. 화면에는 최대 3개다. 토스트가 하나라도 있으면 4초 뒤 setTimeout이 배열의 마지막 항목을 하나 뺀다. 새로 붙인 항목은 앞쪽이므로, 먼저 온 토스트가 뒤에서 빠진다.

문구는 두 가지다. 좋아요는 님이 내 글에 좋아요를 눌렀습니다., 그 외 kind는 님이 내 글에 댓글을 남겼습니다.

토스트나 목록 항목을 누르면 FormData에 notificationId와 postId를 넣고, 화면의 그 항목을 먼저 읽음으로 바꾼 뒤 markNotificationRead를 호출한다. 액션은 세션이 없거나 id가 ObjectId가 아니거나 그 사용자의 행이 아니면 { ok: false }를 반환한다. 성공하고 postId가 24자리 hex이면 redirect("/postdetail/" + postId)한다. 모두 읽음은 화면의 배지를 0으로 만들고 항목을 읽음으로 바꾼 뒤 markAllNotificationsRead를 호출한다. 그 액션은 readAt이 비어 있는 행만 지금 시각으로 갱신한다.

목록에 없는 안 읽은 행이 삭제되면 배지는 그 이벤트로 줄지 않는다. 다음 snapshot이 count를 다시 맞춘다.

참고: MDN setTimeout, MDN FormData, Next.js 서버 액션.

## 7. 로그인 상태가 바뀔 때

isLoggedIn이 바뀌면 effect 안에서 setSnapshot을 호출하지 않는다. 렌더 중에 trackedLogin !== isLoggedIn이면 trackedLogin, 목록, 토스트, 패널을 그 자리에서 맞춘다. 로그인으로 바뀌면 서버가 넘긴 notifications를 쓰고, 로그아웃이면 빈 목록이다. React가 안내하는 방식은, props가 바뀌었을 때 이전 렌더의 상태를 렌더 중에 고치는 것이다.

effect는 로그아웃일 때 seenRef.current = null만 하고 반환한다. ref를 지우는 일은 렌더 밖이다. 로그아웃이거나 notifications가 null이면 메뉴는 null을 반환하고, 스트림 effect는 정리 함수에서 EventSource를 닫는다.

참고: React, effect 없이 상태 맞추기, React useEffect, MDN visibilitychange.

## 8. 이 코드가 하지 않는 일

- 브라우저 알림 권한이나 서비스 워커 푸시로 이 행을 보여 주지 않는다.
- 알림 목록을 "use cache"에 넣지 않는다. 첫 목록은 요청마다 listForUser가 읽고, 그 뒤의 변경은 SSE로 받는다.
- 알림 목록을 5초마다 서버 액션으로 다시 읽지 않는다. actions/notificationActions.ts에는 markNotificationRead와 markAllNotificationsRead만 있다.
- component/organisms/PostLiveRefresh.tsx의 5초 router.refresh는 글 목록과 상세를 다시 그리는 경로다. 알림 메뉴의 조회 간격이 아니다.
- 댓글 삭제 알림은 없다. 이 앱의 댓글 삭제가 notifyAuthor와 연결되어 있지 않다.
- 웹소켓으로 알림을 보내지 않는다.